

game2401 – Game Programming III

Assignment 1: Specification and Foundational Architecture

Student: _____ Date: _____

Code breakdown and suggestions

This is a genuinely strong specification, and it is worth saying so plainly. You map four systems, give each one a single responsibility, and reason SOLID against your own design rather than reciting it, down to the signal that would tell you an abstraction had broken. Each of the three patterns comes with a rationale and a trade-off, which is exactly the level the assignment asks for. The work from here is the gap between that specification and the build, because the foundational code delivers part of the architecture you documented and leaves the rest as a different prototype.

What the build delivers. The interface layer is real and clean. `IIngredient` and `IProcessable` are segregated exactly as you argued, and the Strategy pattern is properly implemented. `ProcessingSystem` holds the three strategies, selects on category, calls `Execute`, and raises `OnIngredientProcessed`. That is the specification made real, and it is good work.

The chemistry core. This is where the build and the document part company. The `ChemistrySystem` that runs is a different design from the one you specified. It pairs ingredients through physics triggers and instantiates a result, while the documented `Evaluate` path is left as a `NotImplementedException`. The central rule, that no raw ingredient can succeed and that the check happens before any lookup, is not in the running code, and the three outcome states are not evaluated there. Decide which design you actually want, then make the document and the build agree on it.

The Observer chain. Your headline pattern, `ChemistrySystem` raising an event that `InventorySystem` consumes, is not wired up. There is no `OnCombinationResolved` event, the subscription inside `InventorySystem` is commented out, and `CombinationLookup` never has its dictionary populated, so the inventory and the lookup are both dead as currently wired. Connecting that chain is the highest-value next step, because it is the part of your own architecture that holds the rest together.

Spec-driven development. The habit to build is keeping the specification and the code in lockstep, so the document never describes a system the build does not run. Two things worth looking into are <https://skillsmp.com> and <https://github.com/github/spec-kit>. It is also worth building yourself a Core patterns folder and writing your own small skills to enforce it, for which Anthropic's Skill Creator is a good starting point: <https://github.com/anthropics/skills/blob/main/skills/skill-creator/SKILL.md>. That gives you a way to hold yourself to your own patterns and working rules as the project grows.

Visualising the systems. Your system map and data flow would read more clearly as diagrams, so learn a little mermaid. It is quick to write, it renders your architecture and event flow at a glance, and it works well when you are planning a system out before you build it.

Tools to look at. The Unity MCP is worth a look, though I have not tested it myself: <https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.0/manual/unity-mcp-overview.html>. And go back through the patterns from class, because several of them map straight onto the systems you are building.

The bar for what comes next. You are clearly working hard with an LLM on this, which is completely fine and good to see. It does raise the bar of what I expect, because that help should let you close the distance between a

strong specification and a build that runs it. Keep that in mind for assignments 2 and 3. The architecture you describe is the architecture I will be looking for in the code.

Where this goes next. Bring the chemistry core and the event chain up to the specification, keep the document honest to the build as you go, and the foundation under all of this is strong. The worked example for this assignment is still worth studying for shape:
<https://sean-teaches-games.vercel.app/game2401/assignments/1/example>

Deliverables

Part 1: Specification

- ☐ Specify the game's intended mechanics and the systems each one demands, including the objects, states, events, and module boundaries.
- ☐ Apply SOLID against the proposed architecture and make the boundaries between systems explicit.
- ☐ Identify the object-oriented design patterns that fit each system, with the rationale and trade-offs considered.

Part 2: Foundational build

- ☐ Produce a runnable build of the foundational systems where the architectural boundaries are visible in the code.
- ☐ Drive the build from the specification, with AI assistance steered by the specification rather than substituted for it.

Part 3: Individual reflection

- ☐ Write a reflection on your development workflow and how you used AI against the specification through the breakdown and the foundational build.

Submission: Submit a zip containing the specification as pdf, a build of the foundational systems, a link to the project repository, and the reflection as pdf.

Rubric

Tick the level reached in each category, then add specifics in the Comments box. Weights are shown per category; enter the final mark at the bottom.

PRODUCTION

Mark: 17.5 / 20

☐	Insufficient	The submitted artefact set is incomplete or non-functional. The specification, the running prototype, and any accompanying tool diverge from one another, with no single artefact serving as the authoritative source. Architectural intent cannot be reconstructed from the materials provided.
☐	Adequate	The artefact set is complete and runs. The specification names the principal objects, states, and events present in the prototype, and the prototype follows that specification at a surface level. Where a tool or toolkit appears, it performs its stated function on the originating project. Documentation records what was built without examining what the build revealed.
☒	Effective	The artefact set shows evidence of reflection during production. The specification has been revised in response to behaviour discovered while

		implementing, and the revisions are visible in the artefact rather than merely claimed. Trade-offs encountered during pattern integration are recorded against the affected modules. Tooling is shaped by friction observed in the workflow it serves.
☐	Accomplished	The artefact set reframes its own scope. The specification reads as the load-bearing source, with the prototype, the custom tool, and the reusable toolkit each derivable from it under the constraints the specification names. Architectural revisions are presented as judgements with stated alternatives, and the toolkit generalises beyond the originating prototype in a form another practitioner could adopt without rewriting it.

Comments

There is real ambition here, Michael, and a specification that sets a high bar for itself. The build runs and the foundation is there, so this is a strong place to be working from. What it needs now is for the document and the code to tell the same story, because the interface and processing layers follow the specification while the chemistry core and the event chain do not yet. The breakdown above is all about closing that distance.

KNOWLEDGEMark: 32.5 / 35

☐	Insufficient	Explanations of pattern selection, specification structure, and AI-collaboration boundaries are missing, contradictory, or restate definitions without reference to the work. The relationship between specification and code is not articulated.
☐	Adequate	Explanations recall the named patterns, specification elements, and AI-workflow stages, and apply them to the present project at the level of identification. Pattern choices are justified by their stated purpose rather than by the situation in which they were selected.
☐	Effective	Explanations relate, compare, and attribute. Pattern selections are reasoned against alternatives that were considered and set aside, with extensibility, performance, and integration cost named as the axes of comparison. Specification-to-code divergence is read as evidence about the design, and AI-generated output is examined against the architectural intent that prompted it rather than accepted on its surface.
☒	Accomplished	Explanations defend the work under counter-cases. The strongest case against the chosen pattern, the chosen specification structure, or the chosen AI-collaboration boundary is constructed and argued from. Trade-offs are stated as commitments rather than caveats, and the boundary between problems suited to AI-assisted authoring and problems requiring direct authorship is reasoned from properties of the problem.

Comments

This is the strongest part of your submission. You reason SOLID against your own systems rather than restating the definitions. You name the signal that would tell you an abstraction had broken, and you give each pattern a rationale and a trade-off. The thing to watch is that your understanding is currently ahead of your build, so the next step is making the reasoning hold in the running code, where the interface-driven path you describe is still a stub.

SKILLSMark: 32.5 / 35

☐	Insufficient	Pattern implementations are incorrect or absent. Specification artefacts and code artefacts are produced as separate documents with no procedural link between them. AI-generated output is committed without examination, or examination is performed without changing what is committed. Performance claims rest on intuition rather than measurement.
☐	Adequate	Pattern implementations follow the canonical structure for the named pattern. The specification is updated when the code is updated, though the order of operations between the two is inconsistent. AI-collaboration steps are executed in the documented sequence, and code produced through that sequence runs. Practitioner vocabulary names the parts present in the work.
☐	Effective	Pattern implementations are selected against the architectural context, with the surrounding modules treated as constraints rather than as background. The specification is treated as the artefact that drives subsequent code generation, and AI-collaboration is operated as a directed workflow in which prompts cite specification sections and outputs are read against them. Performance examination uses measurements taken from the running prototype, with the measurement procedure stated alongside the result. Practitioner vocabulary discriminates between adjacent options.
☒	Accomplished	Pattern implementations adapt under changed requirements without losing their identity, and the adaptations are recorded as specification revisions before they appear in code. AI-collaboration is conducted with the boundary between automated authorship and direct authorship adjusted in response to what each step produces, and divergence between specification and generated code is read as a signal about either the prompt or the specification. Tools constructed during the work are extracted, generalised, and re-applied to a second context within the submission. Practitioner vocabulary is used to reason about the work, not only to label it.

Comments

The Strategy pattern and the segregated interfaces are implemented cleanly, and they do exactly what the document says they do, which is the harder half of this. Where the skill needs to show next is the chemistry core and the Observer chain, both specified well but not yet wired. Build the habit of letting the specification drive the code so the two never drift apart, and the resources in the breakdown above are there to help you work that way.

VALUESMark: 10 / 10

☐	Insufficient	The specification is treated as a deliverable produced once, with subsequent code changes left unreflected in it. AI usage is undisclosed, partially disclosed, or disclosed in terms that obscure what was contributed by hand. Peer review is avoided, deflected, or received without response.
☐	Adequate	The specification is acknowledged as intended to remain current and is updated when prompted by feedback or an explicit checkpoint. AI usage is disclosed in a stated location, with the prompts and the human edits identifiable. Peer review is received and the receipt is recorded.
☐	Effective	The specification is updated when the code drifts from it, without external prompting, and the update is treated as part of the work rather than as overhead. AI usage is disclosed at the granularity at which it was used, including the prompts that did not produce useful output. Peer review is engaged with substantively, with revisions attributable to specific feedback received.
☐	Accomplished	The specification is maintained as the load-bearing artefact across the term, with drift between specification and code treated as a defect to be resolved rather than as expected slippage. AI-collaboration is disclosed with the

		reasoning that selected the boundary, and the disclosure is structured so a reader can audit the boundary. Peer review is engaged as a reciprocal commitment, and reviews offered to others are written to the standard expected of reviews received.
--	--	---

Comments

You are honest in the reflection that the specification moved as you built, and treating the document as something that changes with the work is the right instinct. Hold to that, and take it one step further by keeping the specification current with the code in both directions, so it never describes a system the build does not run. A plan is worth most when it stays true to what is actually there.

Overall feedback

There is strong work here, a real architecture and a clear head for how the systems should fit together. The push from here is to close the gap between the specification and the build, starting with the chemistry core and the event chain, and to keep the document and the code honest to each other as you go. One thing to keep in mind: you are working hard with an LLM, which is good, and it raises the bar of what I will expect in assignments 2 and 3, because that help should let you ship the architecture you can clearly describe. Come find me once the event chain is running, because that is where this gets interesting.

FINAL GRADE: ____ / 100